

Case Study: Building AgentLoom

The problem

An AI agent is simple at its core. It is an AI model that calls functions, looks at the results, and decides what to do next. It repeats this loop until a task is done.

Frameworks like LangChain add a lot of extra code. They include complex memory tools, chains, and event handlers. I wanted to understand the basic loop clearly so I could explain every step in an interview. I built AgentLoom as a small tool calling agent that works without any heavy framework.

Step 1: Naming the states

I mapped out what happens in one turn of an agent before writing any code:

1. The agent asks the AI model what to do next.
2. The model asks to run a tool or says the task is done.
3. The agent runs the tool and checks the output.
4. The agent gives that output back to the model.

This creates four main stages: THINKING, ACTING, OBSERVING, and DONE. If something breaks, it goes to ERROR. If it runs too long, it stops at MAX STEPS.

I built a strict state tracker. The agent can only move to allowed steps. If a bug tries to skip from THINKING straight to OBSERVING, the app stops right away. This catches bugs early instead of creating weird results later.

Step 2: The dispatch table

The agent needs a safe way to run functions. It must not run unsafe system code. I used a simple dictionary that connects text names to real Python functions.

The Tool Registry manages this list. It connects names to functions and runs them. I kept this code very plain. It uses basic Python function calls with try except blocks to handle errors. It does not use heavy validation libraries. This keeps the code clean and easy to read.

Step 3: Parsing what the brain says

The AI model should return clean JSON data, but it often adds extra words or markdown tags around the text. Calling a standard JSON loader directly will cause errors.

I wrote an extract json function to fix this. First, it removes markdown code tags. Next, it looks for the first opening bracket and tracks the text until it finds the matching

closing bracket. Then it parses that block as JSON.

If the AI produces broken data, the agent stops and moves to an ERROR state. It is better to stop than to guess wrong and run the wrong command.

Step 4: Making it doable without an API key

Testing an agent usually requires a paid API key. I wanted anyone to test this app for free without setting up account keys.

I built a demo planner as a stand in. It uses simple text rules to solve basic tasks. It can handle basic math, tell the time, or count words. It can also link two tool calls together in a row.

This shows the main loop working correctly without spending money. You can replace the demo planner with a real AI model later by changing one line of code.

Trade-offs I made on purpose

- One tool call at a time. The agent runs one function per step. This is slower, but it makes the task log easy to read and test.
- No retries on bad JSON. If the model sends bad formatting, the app stops instead of asking the model to try again.
- No saved memory. Every run starts fresh. Past runs do not affect new tasks.

What I would improve with more time

- Ask the model to fix its text if the JSON parsing fails.
- Allow the agent to run multiple tools in a single turn.
- Add direct support for real AI models like OpenAI using the same setup.
- Stream live updates to the web dashboard while the agent runs.

What I learned

Building this loop by hand taught me how simple AI agents really are. An agent is just a loop with a function dictionary, a JSON parser, and a state tracker. You do not always need a large framework to build smart tools.

AgentLoom

Give it a task. Watch it think, call tools, and observe results until it's done.

1. Give it a task

What is 12 plus 30, then multiply the result by 2?

What is 12 plus 30, then multiply the resu...

What is the current time?

How many words are in "the quick brown fox..."

Write me a poem about the ocean.

Max steps:

8

Run agent

2. Registered tools

add — Add two numbers together.

multiply — Multiply two numbers together.

get_time — Return the current UTC time as a string.

word_count — Count how many words are in a piece of text.

3. Trace

done The result is 84.0.

THINKING decided to call add with {"a":12,"b":30}

ACTING calling add with {"a":12,"b":30}

OBSERVING add returned 42

THINKING decided to call multiply with {"a":42,"b":2}

ACTING calling multiply with {"a":42,"b":2}

OBSERVING multiply returned 84

THINKING decided the task is finished

DONE final answer: The result is 84.0.

AgentLoom: A project demonstrating a tool-calling agent loop, built completely without an agent framework.

AgentLoom

Give it a task. Watch it think, call tools, and observe results until it's done.

1. Give it a task

What is the current time?

What is 12 plus 30, then multiply the resu...

What is the current time?

How many words are in "the quick brown fox..."

Write me a poem about the ocean.

Max steps: 8

Run agent

2. Registered tools

add — Add two numbers together.

multiply — Multiply two numbers together.

get_time — Return the current UTC time as a string.

word_count — Count how many words are in a piece of text.

3. Trace

done The result is 2026-09-25 18:10:57 UTC.

THINKING decided to call `get_time` with {}

ACTING calling `get_time` with {}

OBSERVING `get_time` returned "2026-09-25 18:10:57 UTC"

THINKING decided the task is finished

DONE final answer: The result is 2026-09-25 18:10:57 UTC.

AgentLoom: A project demonstrating a tool-calling agent loop, built completely without an agent framework.

AgentLoom

Give it a task. Watch it think, call tools, and observe results until it's done.

1. Give it a task

How many words are in "the quick brown fox jumps over the lazy dog"?

What is 12 plus 30, then multiply the resu...

What is the current time?

How many words are in "the quick brown fox..."

Write me a poem about the ocean.

Max steps: 8

Run agent

2. Registered tools

add — Add two numbers together.

multiply — Multiply two numbers together.

get_time — Return the current UTC time as a string.

word_count — Count how many words are in a piece of text.

3. Trace

done The result is 9.

THINKING decided to call `word_count` with {"text": "the quick brown fox jumps over the lazy dog"}

ACTING calling `word_count` with {"text": "the quick brown fox jumps over the lazy dog"}

OBSERVING `word_count` returned 9

THINKING decided the task is finished

DONE final answer: The result is 9.

AgentLoom: A project demonstrating a tool-calling agent loop, built completely without an agent framework.